



U-15 長野 プログラミング コンテスト



講習の内容

- U-15 長野プログラミングコンテストについて
- プログラミングの基本学習
- 補足資料



U-15 長野
プログラミング
コンテスト

U15長野プログラミングコンテストについて



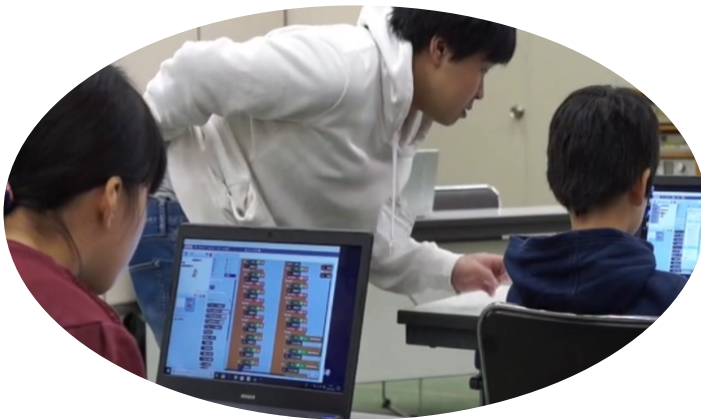
U-15長野
プログラミング
コンテスト

U-15長野プログラミングコンテストとは

2018年から開催された15歳以下を対象とした
プログラミングのコンテストです。

対戦形式のゲーム（CHaser）で勝敗を競います。

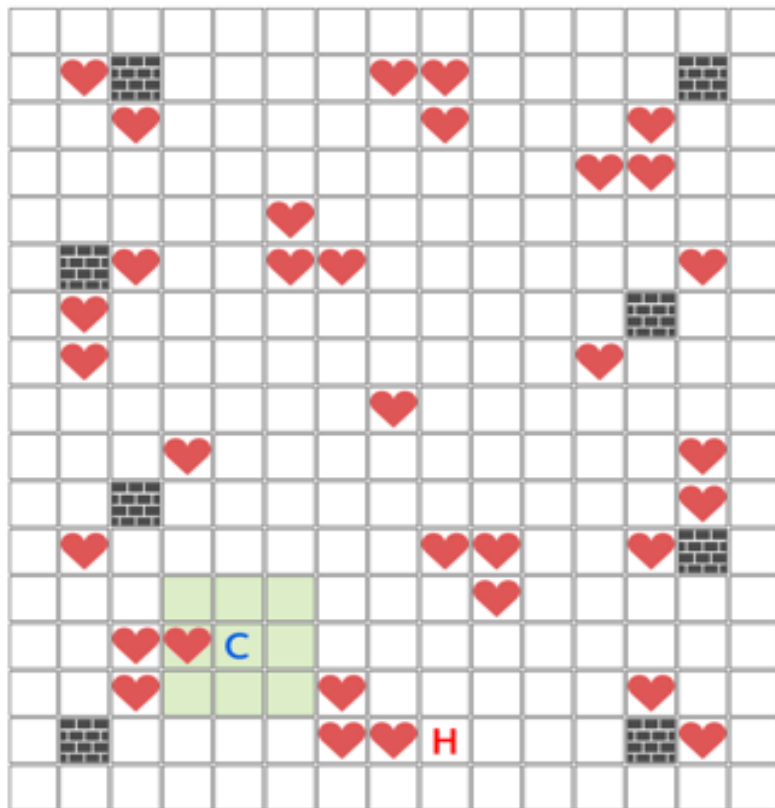
事前講習会は、信州大学工学部・長野工業高等専門学校・長野工業
高等学校の学生が講師となり、プログラミングを教えます。





コンテストの内容

CHaser (チェイサー)

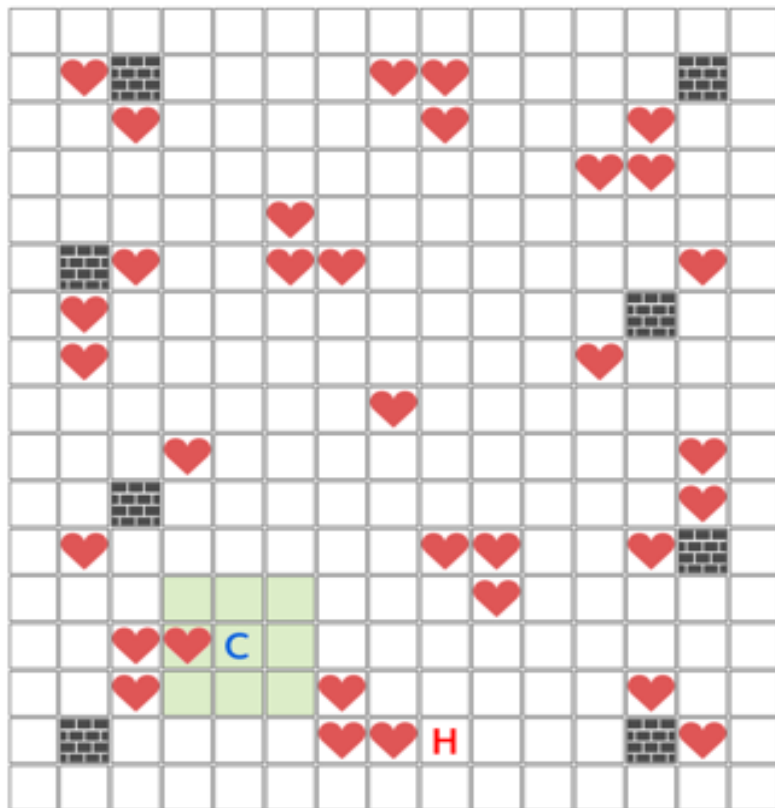


15×17マスの盤面上で、
2人のプレイヤーが1対1で対戦する
ゲーム。

プレイヤーCoolとプレイヤーHotが
互いに攻撃し合いながら、
アイテムを取り合うターン制ゲーム。
(所定のターン数終了まで)

コンテストの内容

CHaser (チェイサー)



命令は全部で **4 種類**

- ・ 1 マス移動 (WALK)
- ・ 周囲確認① (LOOK)
- ・ 周囲確認② (SEARCH)
- ・ ブロックを置く (PUT)



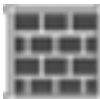
．．． プレイヤー 1



．．． プレイヤー 2



．．． アイテム



．．． ブロック



コンテストの内容

CHaser (チェイサー)

(ゲーム動画)



ゲームサーバ

CHaserを実行する①

事前に案内したプログラムを使います。

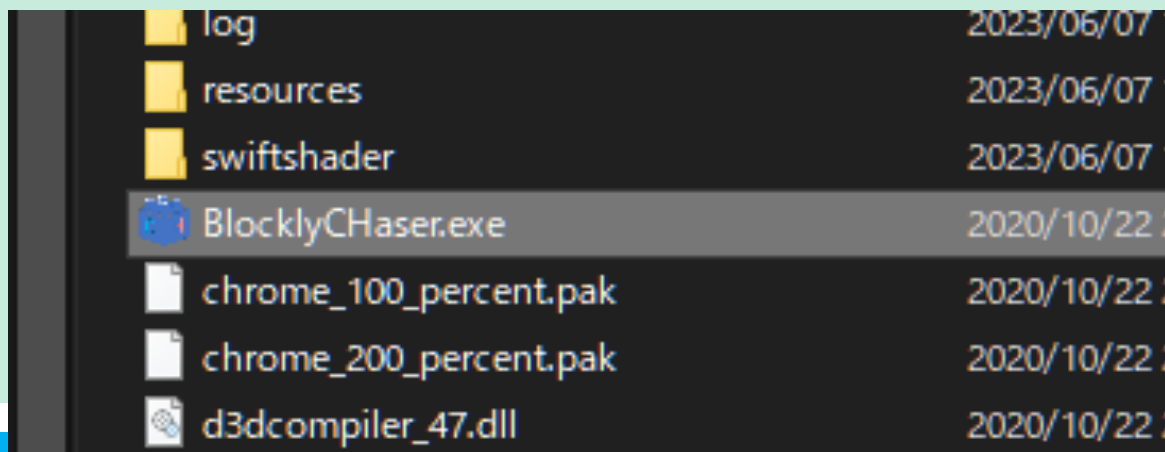
- ① Zipファイル（Csharp_server.zip）のアクセス許可
- ② Zipファイル（Csharp_server.zip）の解凍
- ③ プログラムの実行
- ④ ゲームの開始



ゲームサーバ

CHaserを実行する②

展開されたフォルダ内の「BlocklyCHaser.exe」というファイルをダブルクリックで実行すると、ゲームサーバを起動することができます。



ゲームサーバ

CHaserを実行する③

サーバが開いたら、
「対戦」をクリック
します。



ゲームサーバ

CHaserを実行する④

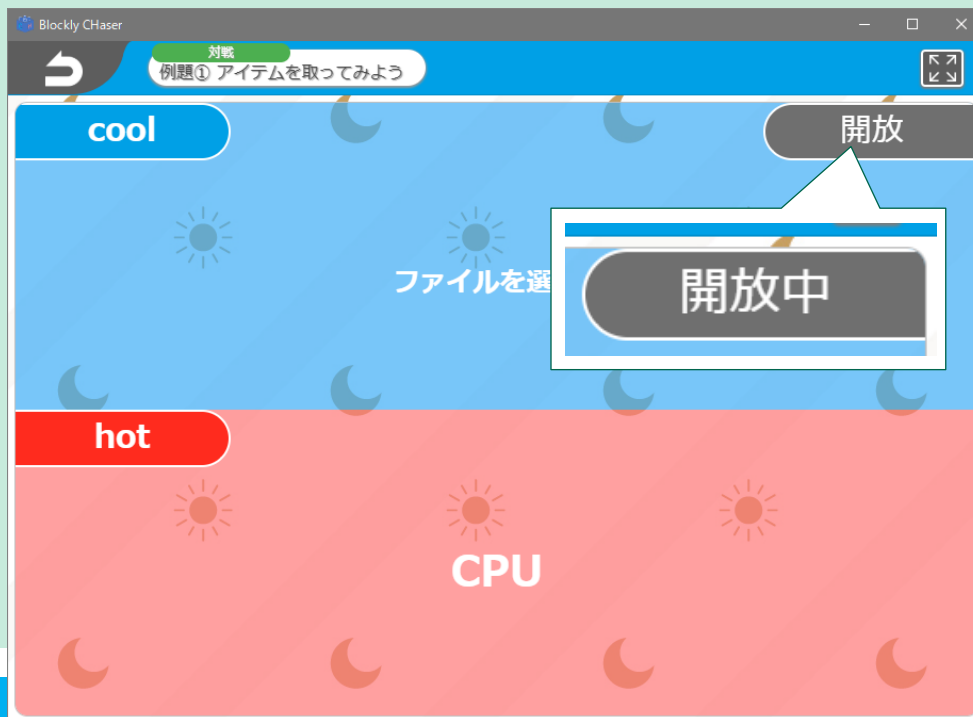
例題①を選択して
「ルーム決定」を
クリックしましょう。



ゲームサーバ

CHaserを実行する⑤

「開放」を一度
クリックして、
表示を「開放中」に
切り替えます。





プレイヤープログラム

プログラムを開く①

事前に案内したプログラムを使います。

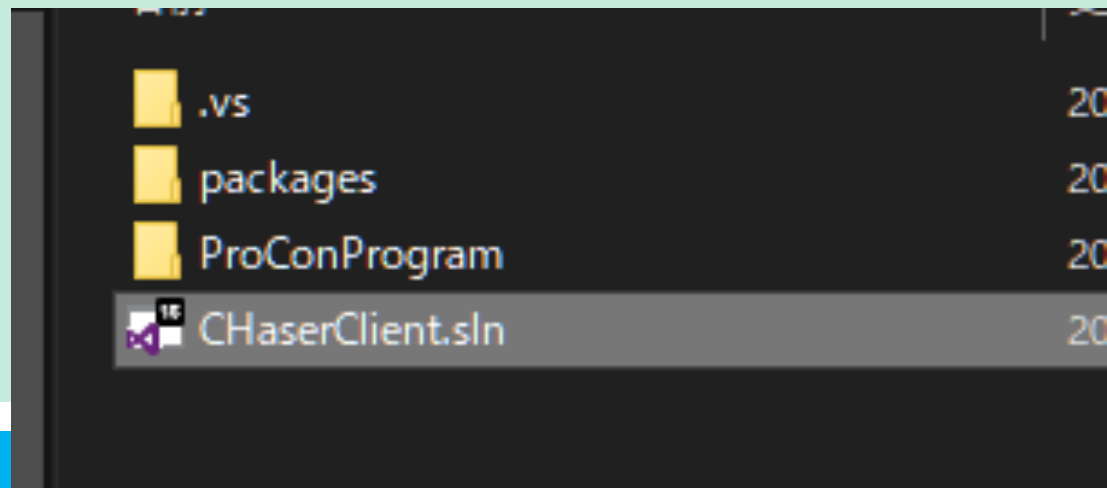
- ① Zipファイル（Csharp_player.zip）のアクセス許可
- ② Zipファイル（Csharp_player.zip）の解凍
- ③ プログラムの起動
- ④ ソースコードの確認



プレイヤープログラム

プログラムを開く②

展開されたフォルダ内の「CHaserClient.sln」というファイルをダブルクリックで実行するとプログラムを確認することができます。





プレイヤープログラム

プログラムを開く③

プログラムが開いたら、赤枠の一文を書き加えてみましょう。

```
while (true)
{
    GetReady命令
    result = Target.WalkRight();
```



プレイヤープログラム

プログラムを開く④

プログラムを書き足したら、実行してみましょう。
画面上部の「▶開始」をクリックします。





プレイヤープログラム

CHaserを実行する⑥

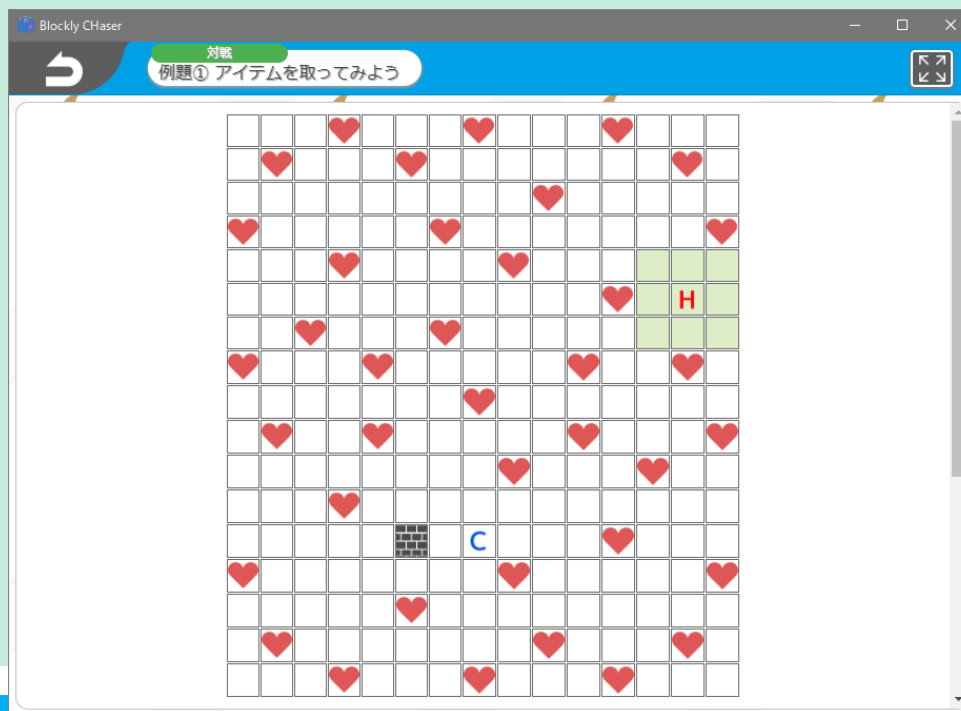
先程のゲームサーバの
画面に戻り、
「ゲームスタート」を
クリックしてみましょう。



プレイヤープログラム

CHaserを実行する⑦

ゲームが開始されれば
成功です。





U-15 長野
プログラミング
コンテスト

プログラミングの基本学習



命令の種類と実行順序

命令の種類（待機）

- ・ 待機する：GetReady
 - 自分のターンが来るのを待つ。
 - 自分のターンが来たら、周囲のマス情報を取得する。



命令の種類と実行順序

命令の種類（行動）

- ・ 移動する : Walk (WalkUp, WalkDown, WalkRight, WalkLeft)
- ・ 探す : Search (SearchUp, SearchDown, SearchRight, SearchLeft)
- ・ 探す : Look (LookUp, LookDown, LookRight, LookLeft)
- ・ ブロックを置く : Put (PutUp, PutDown, PutRight, PutLeft)



命令の種類と実行順序

命令の実行順序（例）

待機命令

行動命令

1 ターン目

待機命令

行動命令

2 ターン目

待機命令

行動命令

3 ターン目



命令の種類と実行順序

命令の実行順序（例）

GetReady

WalkUp

1 ターン目

GetReady

WalkLeft

2 ターン目

GetReady

LookLeft

3 ターン目



コードを書いてみよう

ターンの開始

行動開始前に、必ずGetReady命令を実行します。

この命令が実行されると、

value[0]

周辺情報がvalue配列（＊）に格納されます。

アイテムがある → result[0]はItem。

ブロックがある → result[0]はBlock。

何もない → result[0]はNothing。

＊ 「配列」 は後述します。

0	1	2
3	C	5
6	7	8



コードを書いてみよう

共通の書き方（行動系）

- 命令の書き方

result = Target.**[実行する命令]**();

※ 実行結果がresultという配列に格納されます。

- Walkの場合は、移動後の周辺情報
- Lookの場合は、指定方向3×3マスの情報



コードを書いてみよう

Walk命令の種類

Walk命令は 4 種類あります。

命 令	動 き
WalkUp	上に 1 マス移動する
WalkRight	右に 1 マス移動する
WalkLeft	左に 1 マス移動する
WalkDown	下に 1 マス移動する



Walk

例. WalkUp (上への移動)

<書き方>

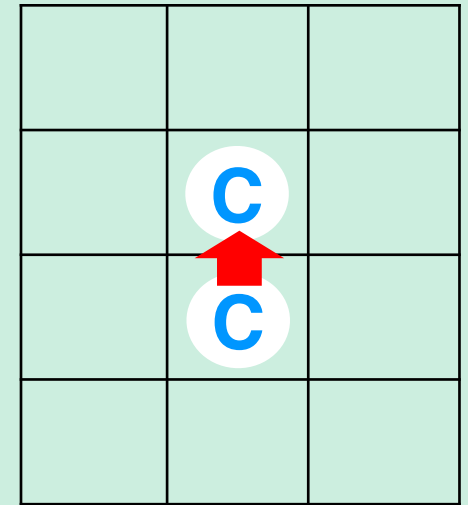
```
result = Target.WalkUp();
```

<動き>

1. 上のマスに移動します。
2. 移動した後の自分の周辺情報が

resultに取得されます。

(命令が1回実行された扱いになり、このターンには他の命令は実行できません)



Walk命令を実行した後のresult

移動した後の周囲の情報がresultに入る。

例. result[0] には移動した後の
マスの左上の情報が入っている。

- アイテムがある → result[0]はItem。
- ブロックがある → result[0]はBlock。
- 相手プレイヤーがいる → result[0]はPlayer。
- 何もない → result[0]はNothing。

result[0]

0	1	2
3	C	5
6	C	8



コードを書いてみよう

命令の種類

Look命令は 4 種類あります。

命 令	動 き
LookUp	上3×3マスの情報を取得する
LookRight	右3×3マスの情報を取得する
LookLeft	左3×3マスの情報を取得する
LookDown	下3×3マスの情報を取得する



Look

例. LookUp (上を調べる)

<書き方>

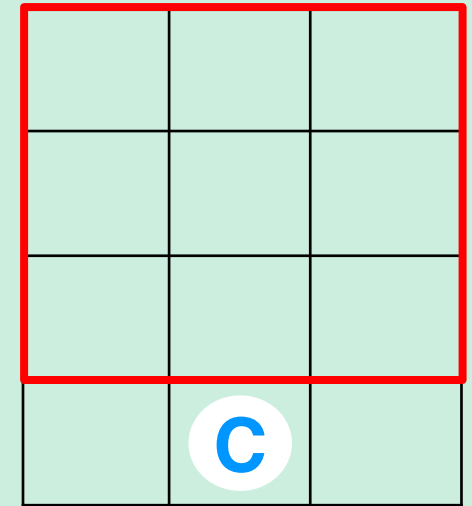
```
result = Target.LookUp();
```

<動き>

1. 上3×3マスの情報を取得します。
2. 取得した計9マスの情報が

resultに取得されます。

(命令が1回実行された扱いになり、このターンには他の命令は実行できません)





Look命令を実行した後のresult

Lookした方向3×3マスの情報がresultに入る。

例. result[7] には頭上のマスの
情報が入っている。

アイテムがある → result[7]はItem。
ブロックがある → result[7]はBlock。
相手プレイヤーがいる → result[7]はPlayer。
何もない → result[7]はNothing。

result[0]

0	1	2
3	4	5
6	7	8
	C	



コードを書いてみよう

命令の種類

Search命令は 4 種類あります。

命 令	動 き
SearchUp	上に直線上9マスの情報を取得する
SearchRight	右に直線上9マスの情報を取得する
SearchLeft	左に直線上9マスの情報を取得する
SearchDown	下に直線上9マスの情報を取得する



Search

例. SearchLeft (左を調べる)

<書き方>

```
result = Target.SearchLeft();
```

<動き>

1. 左の直線状9マスの情報を取得します。
2. 取得した計9マスの情報がresultに取得されます。

(命令が1回実行された扱いになり、このターンには他の命令は実行できません)





Look命令を実行した後のresult

Searchした方向の直線状9マスの情報がresultに入る。

例. result[0] には左のマスに入っている。

アイテムがある → result[0]はItem。

ブロックがある → result[0]はBlock。

相手プレイヤーがいる → result[0]はPlayer。

何もない → result[0]はNothing。



result[0]



コードを書いてみよう

命令の種類

Put命令は4種類あります。

命 令	動 き
PutUp	上にブロックを置く
PutRight	右にブロックを置く
PutLeft	左にブロックを置く
PutDown	下にブロックを置く



Put

例. PutUp (上へブロックを置く)

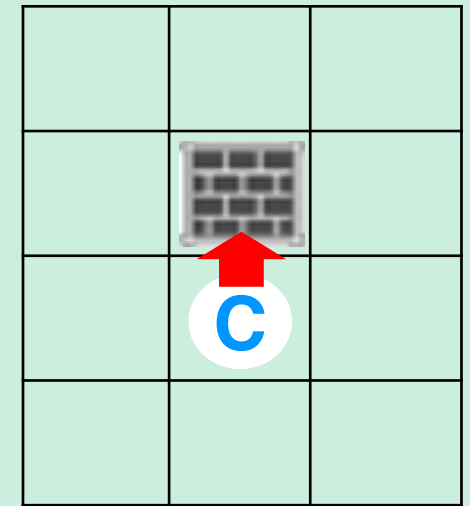
<書き方>

```
result = Target.PutUp();
```

<動き>

1. 上のマスにブロックを置きます。
2. ブロックを置いた後の自分の周辺情報が
resultに取得されます。

(命令が1回実行された扱いになり、このターンには他の命令は実行できません)



Put命令を実行した後のresult

ブロックを置いた後の
周囲の情報がresultに入る。

例. result[1] には頭上のマス
の情報が
入っている。

Put命令で頭上にブロックを置いた後なので
result[1]はBlock。

result[1]		
0		2
3	C	5
6	7	8



コードを書いてみよう

命令を実行する①

さきほど書いたWalkRight命令の動きをもう一度見てみましょう。

コードには、

WalkRight命令のみが

記載されています。

```
while (true)
{
    GetReady命令
    result = Target.WalkRight();
}
```



コードを書いてみよう

命令を実行する②

自分のターンになると、
右に 1 マス移動します。

このままでは右側の壁に
衝突してしまいます。

条件分岐の構文を
学んでいきましょう。





コードを書いてみよう

条件分岐の構文

C#言語の条件分岐には、

- ・ if構文
- ・ switch構文

などがあります。

```
if (value[Up] == Item)
{
    result = Target.WalkUp();
}
else if (value[Left] == Item)
{
    result = Target.WalkLeft();
}
else if (value[Right] == Item)
{
    result = Target.WalkRight();
}
else if (value[Down] == Item)
{
    result = Target.WalkDown();
}
else
{
    result = Target.LookUp();
}
```




コードを書いてみよう

if構文

if (条件式①)

{

条件式①が成立したときに実行される命令

}

else if (条件式②)

{

条件式②が成立したときに実行される命令

}



コードを書いてみよう

if構文

- 1個目は必ず if。
- else if は if または else if の後にのみ使うことができる。
if または 上位の else if が成立した場合は読み飛ばされる。
- else if は何個あっても良い。
- else を使う場合は、一番最後に 1 回のみ使うことができる。



コードを書いてみよう

if構文のサンプル①

右のコードは、
自分の周りにアイテムがあったら
取得し、アイテムが無ければ
上方向にLookする
コードサンプルです。

```
if (value[Up] == Item)
{
    result = Target.WalkUp();
}
else if (value[Left] == Item)
{
    result = Target.WalkLeft();
}
else if (value[Right] == Item)
{
    result = Target.WalkRight();
}
else if (value[Down] == Item)
{
    result = Target.WalkDown();
}
else
{
    result = Target.LookUp();
}
```



コードを書いてみよう

条件式で使う比較演算子

演算子	意 味
==	左右が同じものである
!=	左右が違うものである
<	右のものは左のものよりも大きい
<=	右のものは左のもの以上
>	左のものは右のものよりも大きい
>=	左のものは右のもの以上



コードを書いてみよう

if構文のサンプル②

右のコードは、
ブロックの無い方向に移動する
コードサンプルです。
elseのブロックが無いので、
未完成のように思われますが
実際には4方向すべてブロック
であることはあり得ないので、
問題はありません。

```
if (value[1] != Block)
{
    result = Target.WalkUp();
}
else if (value[3] != Block)
{
    result = Target.WalkLeft();
}
else if (value[5] != Block)
{
    result = Target.WalkRight();
}
else if (value[7] != Block)
{
    result = Target.WalkDown();
}
```



コードを書いてみよう

if構文のサンプル③

条件式を2つ以上記述するif構文のサンプル

複数の条件式を「&&」（意味：かつ）、「||」（意味：または）で繋ぐと、複数の条件を判定できるようになります。

```
if (value[1] != Block && value[0] != Player && value[2] != Player)
{
    result = Target.WalkUp();
}
```



コードを書いてみよう

switch構文

```
switch (value[1])  
{  
    case Player:  
        value[1]がPlayerだったときに実行される命令  
        break;  
    case Block:  
        value[1]がBlockだったときに実行される命令  
        break;  
    case Item:  
        value[1]がItemだったときに実行される命令  
        break;  
    default  
        それ以外の場合に実行される命令  
        break;  
}
```



コードを書いてみよう

switch構文のサンプル

```
switch (value[1])  
{  
    case Player:  
        result = Target.PutUp();  
        break;  
    case Block:  
        result = Target.WalkRight();  
        break;  
    case Item:  
        result = Target.WalkUp();  
        break;  
    default:  
        result = Target.WalkUp();  
        break;  
}
```

頭上のマスが、

- ・ 相手プレイヤーだったら
→ ブロックを置く。
- ・ ブロックだったら
→ 右に移動する。
- ・ アイテムだったら
→ 上に移動して取る。
- ・ それ以外の場合
→ 上に移動する。



例題①

制限時間 30分

これまでの内容を活用して、例題に挑戦してみましょう。

1. アイテムを3つ取ることを目標にしてみましょう。
2. 最後まで生き残ることを目標にしてみましょう。

プログラムが思うように動かないときは、
「デバッグ実行」をしてみましょう！
(参考. p.73 「デバッグ実行」)



例題 ②

制限時間 20分

これまでの内容を活用して、例題に挑戦してみましょう。

1. 相手プレイヤーにPUTして勝ちましょう。

接続先を「02_practice」に変更して実行します。

```
//使用するマップとプレイヤー名の指定  
Target.Name = "自分";  
Target.Room_Name = "02_practice";
```



演習問題 ①

制限時間 -- 分

これまでの内容を活用して、演習問題に挑戦してみましょう。

1. アイテムを3つ以上取る
2. 最後まで生き残る

接続先を「03_practice」に変更して実行します。

```
//使用するマップとプレイヤー名の指定  
Target.Name = "自分";  
Target.Room_Name = "03_practice";
```



演習問題 ②

制限時間 -- 分

これまでの内容を活用して、演習問題に挑戦してみましょう。

1. 最後まで生き残る

接続先を「04_practice」に変更して実行します。

```
//使用するマップとプレイヤー名の指定  
Target.Name = "自分";  
Target.Room_Name = "04 practice";
```



演習問題 ③

制限時間 -- 分

これまでの内容を活用して、演習問題に挑戦してみましょう。

1. ボットプログラムに勝利する

接続先を「05_practice」に変更して実行します。

```
//使用するマップとプレイヤー名の指定  
Target.Name = "自分";  
Target.Room_Name = "05_practice";
```



ボットプログラムとの対戦方法

ボットプログラムの起動

事前に案内したプログラムを使います。

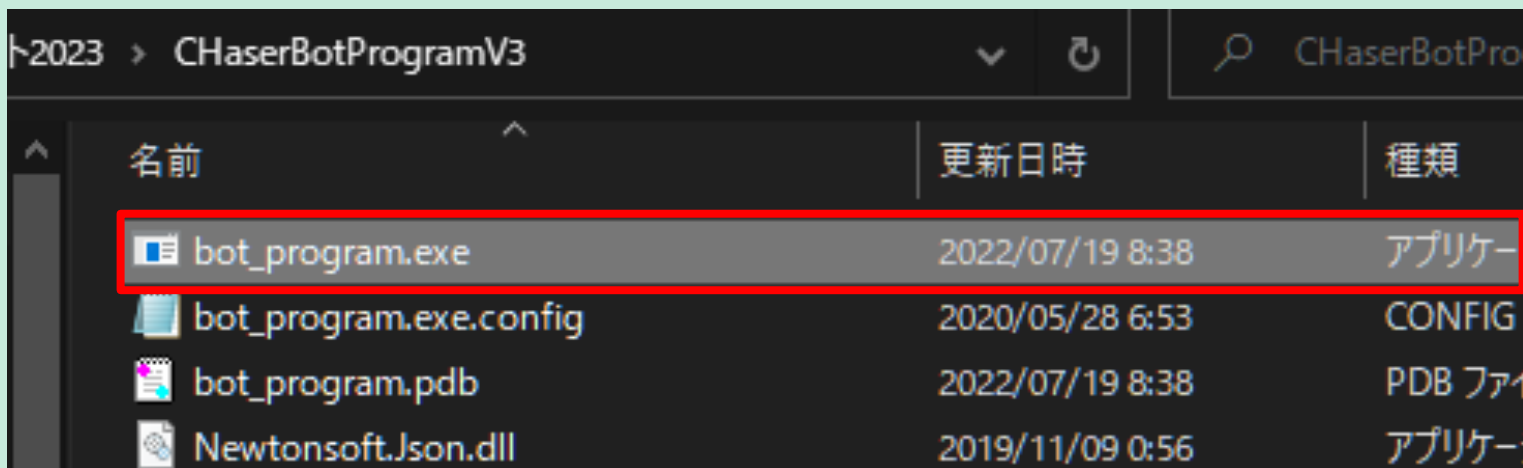
- ① Zipファイル（Csharp_bot.zip）のアクセス許可
- ② Zipファイル（Csharp_bot.zip）の解凍
- ③ プログラムの起動



ボットプログラムとの対戦方法

ボットプログラムの起動

「ChaserBotProgramV3」フォルダ内の「bot_program.exe」です。



名前	更新日時	種類
bot_program.exe	2022/07/19 8:38	アプリケー
bot_program.exe.config	2020/05/28 6:53	CONFIG
bot_program.pdb	2022/07/19 8:38	PDB ファイ
Newtonsoft.Json.dll	2019/11/09 0:56	アプリケー



演習問題 ④ ～

制限時間 -- 分

この他にも様々なマップを用意しましたので
挑戦してみましょう。

また、自分のマップを作成して
プログラムの動きを確認してみましょう。



U-15 長野
プログラミング
コンテスト

補 足 資 料 (大会ルール)



ルール説明

基本ルール

予選大会のルール①

- 運営が用意したボットプログラムと対戦する。
プレイヤーは必ず先攻。
- 1人2回、対戦を行う。
1回目と2回目のマップは変わる。
- ターン数は100ターン。



ルール説明

基本ルール

予選大会のルール②

- ゲーム終了時、得点を計算する
 - ✓ 取得したアイテム数 × 3 点
 - ✓ PUTで勝った場合、残りターン数を得点に加算
 - ✓ PUTで負けた場合、または自滅した場合
残りターン数を点数から減算



ルール説明

基本ルール

マップの制約

- 全部で3パターンのマップがある（「競技部門運営細則」参照）。
- プレイヤー、アイテム、ブロックの配置は点対称
- アイテムを取ったとき、元々いたマスはブロックに変わる。
- 開始時、上下の9マス目にはブロックは存在しない。



ルール説明

基本ルール

決勝大会のルール①

- 予選大会を勝ち上がった上位4名（※）がトーナメント形式でプレイヤー同士が対戦する。
- 先行／後攻を入れ替えて同じマップで2回対戦する。
- ターン数は100ターン。

※ 昨年度は小学生の部4名、中学生の部4名が決勝に進出。



ルール説明

基本ルール

決勝大会のルール②

- 勝った回数が多いプレイヤーの勝利
- 勝った回数が同じである場合、
引き分け処理を行う（「競技部門運営細則」参照）
- それでも決着がつかない場合、
マップを変更して再戦を行う



U-15 長野
プログラミング
コンテスト

補 足 資 料 (プログラミング言語学習)



C#言語

変数とデータ型

「変数」とは

データを入れる箱のようなもの。「決まった形式」のデータを入れることができ、また、既に入ったデータとは別のデータを代入することもできる。変数には予約語（※）を除き好きな名前を付けることができる。数学で使う変数 x や y とほとんど意味は同じ。

※ プログラム言語であらかじめ定められている、
特定の意味で使われる単語（例. 「if」 など）



C#言語

変数とデータ型

「決まった形式」とは
コンピュータがデータを扱うためにあらかじめ定められた
データの種類（「データ型」という）。

「データ型」の一覧を次のスライドで示します。



C#言語

変数とデータ型

データ型	データの種類
int	32bit整数。 (-2,147,483,648 ~ 2,147,483,647)
long	64bit整数。 (-9,223,372,036,854,775,808 ~ 9,223,372,036,854,775,807)
double	小数を含む数字。
string	文字。
bool	true または false。



C#言語

変数とデータ型

変数を使う時は「これからsuujiという名前のint型の変数をコードの中で使いますよ」ということを明記（「変数の宣言」）して、コンピュータに変数を認識させておく必要があります。

「変数の宣言」と同時に、コンピュータの中のメモリという記憶領域に、その変数のデータを保存するだけの領域があらかじめ確保されます。

次のスライドに、宣言と代入の書き方の例を示します。



C#言語

変数とデータ型

```
7
8  long suuji_long = 567890123456;
9
10 string moji = "右のマスにはヤツがいる";
11
12 double suuji_double = 4.55;
13
14 bool kekka = true;
15
```



C#言語

変数とデータ型

変数の中にあるデータを書き換える行為を変数にデータを「代入する」と言います。

この言い方は、数学という学問における「代入」と同じです。

(書き方) `suuji = 100`

※「左の変数に右の数を代入する」となります。

左右を逆に書くことはできません。

次のスライドに代入の書き方の例を示します。



C#言語

変数とデータ型

```
39  
40  
41  suuji_long = 998877665544;  
42  
43  moji = "左のマスには何もない";  
44  
45  suuji_double = -0.123;  
46  
47  kekka = false;  
48  
49
```



C#言語

変数とデータ型

四則演算などを用いて計算を行い、その計算結果を変数に代入することもできます。

```
suuji = suuji / 100
```

演算子	用途
+	A + B (AとBの和)
-	A - B (AからBを引いたときの差)
*	A * B (AとBの積)
/	A / B (AをBで割ったときの商)
%	A % B (AをBで割ったときの余)



C#言語

配 列

複数の変数をもつ変数のことです。たくさんの変数を一度に宣言して使うことができます。

```
int[] value = new int[9]; //要素数が9個の配列を宣言
```

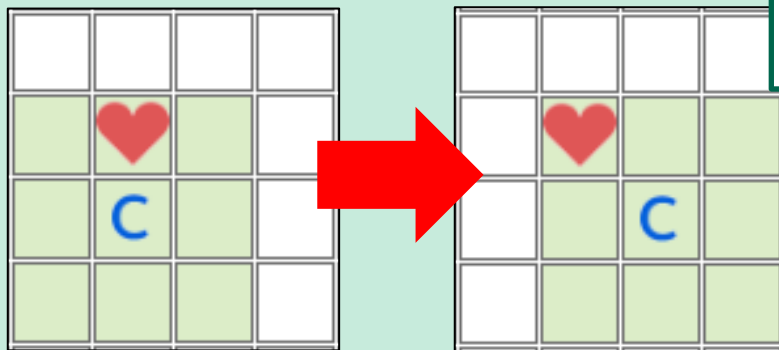
実は今回使用しているvalueやresultはint型の配列です。

上のように9つのint型の変数からなる配列valueは、value[0]、value[1]、value[2]、.....、value[8] はそれぞれがint型の変数です。

Visual Studioの使い方

デバッグ実行

プログラムが思ったように動かないとき、
どの部分に問題があるかを特定する方法として「デバッグ実行」があります。

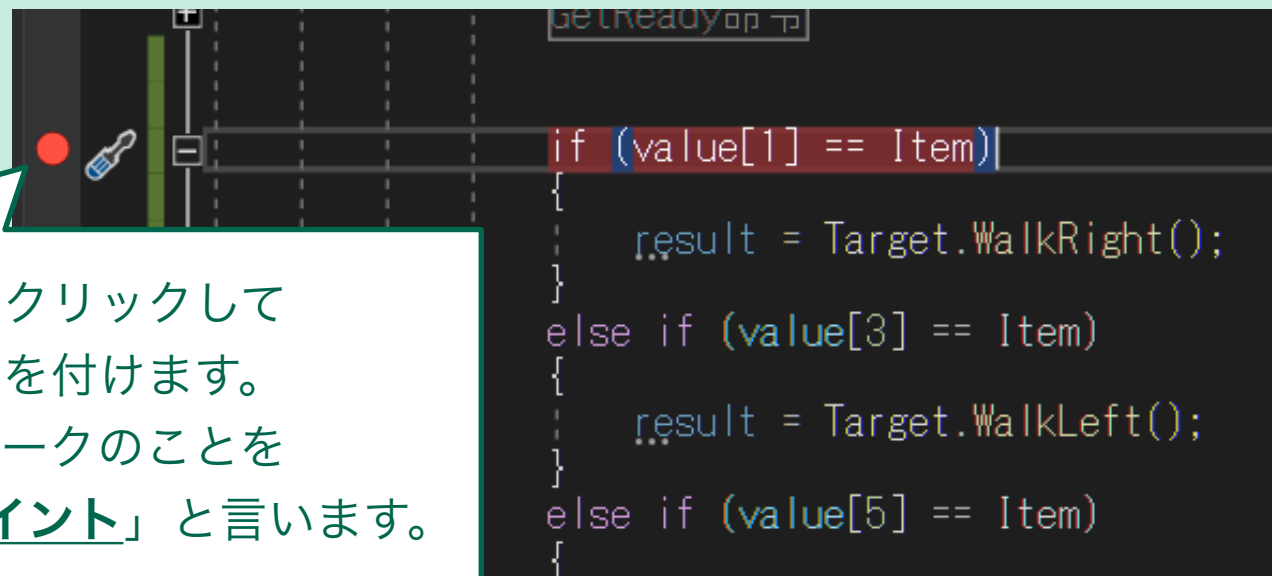


アイテムがあったのに
素通りしてしまった。。

Visual Studioの使い方

デバッグ実行

行動を開始する前のコードにマークを付けます。



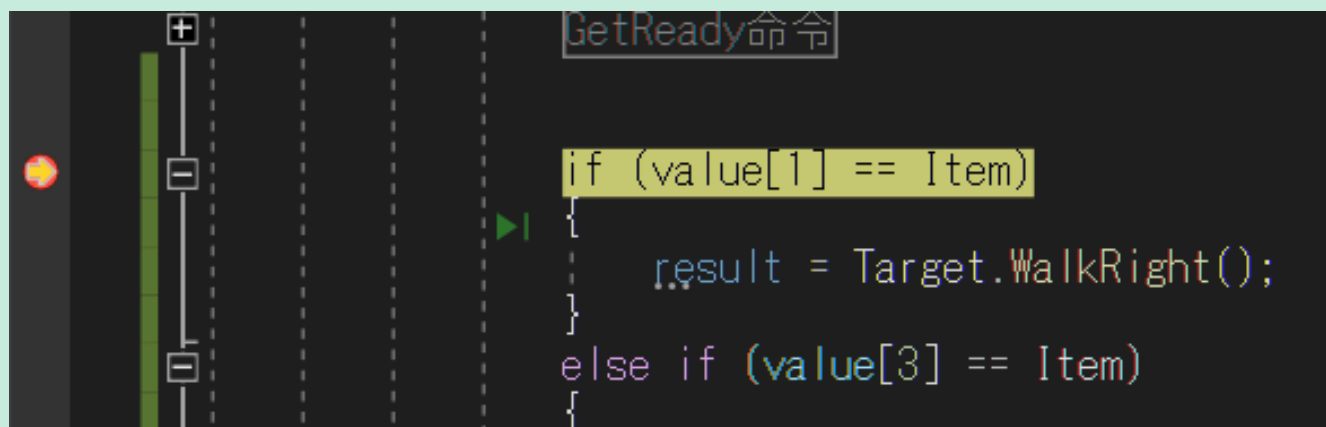
ここをクリックして
マークを付けます。
このマークのことを
「ブレークポイント」と言います。



Visual Studioの使い方

デバッグ実行

プログラムを実行する（ゲームを開始する）と、
指定したブレークポイントでプログラムの実行が止まります。





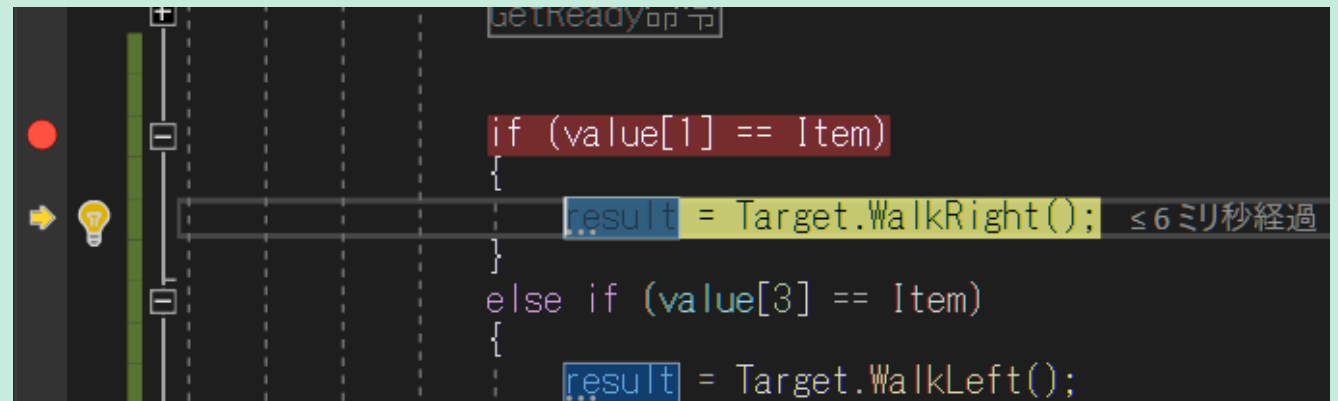
Visual Studioの使い方

デバッグ実行

F10キーを押すと、

次のステップのコードが実行（ステップ実行）されます。

F5キーを押すと、次のブレークポイントまでコードが実行されます。



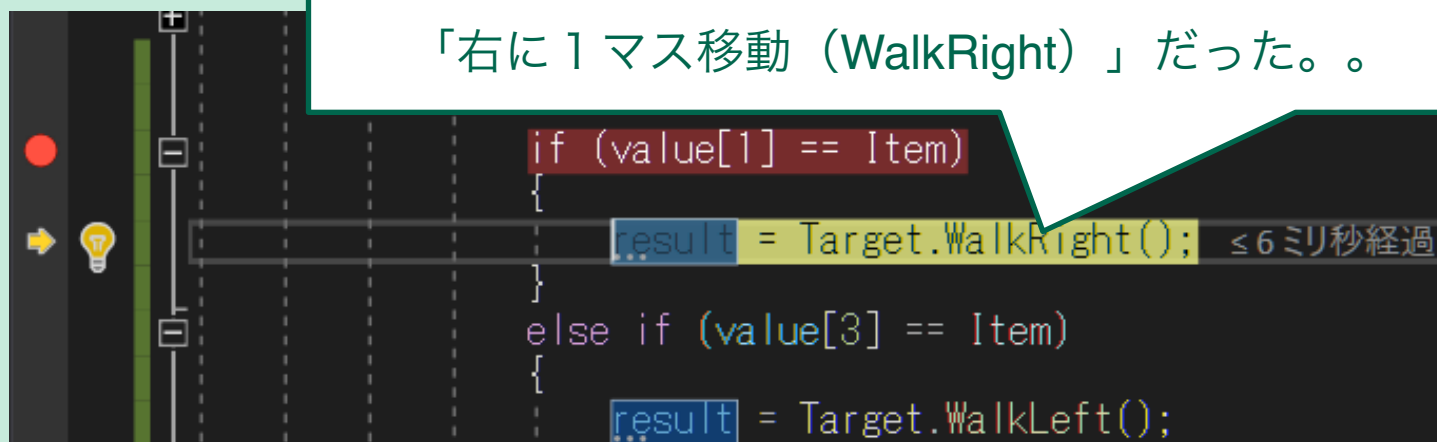


Visual Studioの使い方

デバッグ実行

これらを組み合わせてコードの不具合を探っていきます。

上にアイテムがあったときに実行されるコードが
「右に1マス移動 (WalkRight)」だった。。





技術仕様 (Blockly_CHaser_Tool.html)

Blockly_CHaser_Tool.html

マップを作成するためのツールです。
ゲームサーバのフォルダ内にあります。

ファイルは以下の通り。

CHaserGameServer

→ Blockly-CHaser-Tool-master

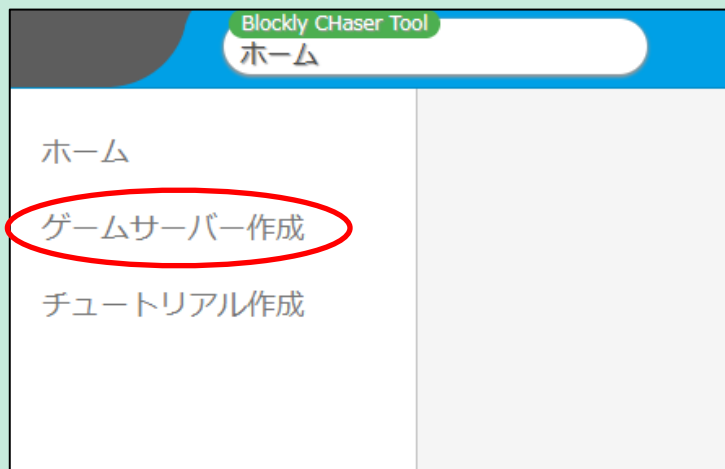
→ Blockly_CHaser_Tool.html



技術仕様 (Blockly_CHaser_Tool.html)

Blockly_CHaser_Tool.html

- ① 「ゲームサーバー作成」を選択します。



- ② 「サーバー名」と「サーバーID」を入力します。

サーバー名	
サーバーID	半角英数字



技術仕様 (Blockly_CHaser_Tool.html)

Blockly_CHaser_Tool.html

次の例では、「サーバー名」は「例題① アイテムを取ってみよう」
「サーバーID」は「01_practice」です。

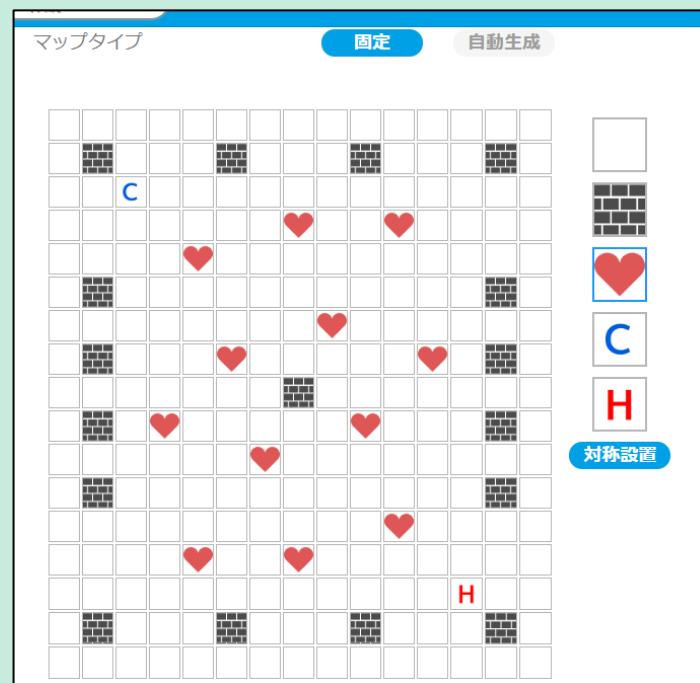


技術仕様 (Blockly_CHaser_Tool.html)

Blockly_CHaser_Tool.html

- ③ マップ上にアイテムやブロックを置いて
マップを作成します。

※ 大会で使われるマップには
いくつかの作成要領がありますので
ご注意ください。





技術仕様 (Blockly_CHaser_Tool.html)

Blockly_CHaser_Tool.html

④ 対戦相手の設定

- ・ CPUモード : [ON] コンピュータとの対戦、[OFF] 人との対戦
- ・ CPUターン : [COOL] コンピュータが先攻、[HOT] コンピュータが後攻
- ・ CPUレベル :

[0] ... Lookするだけ

[1] ... 適当に動き回る

CPUモード	☐ ON	☒ OFF
CPUターン	☒ COOL	☐ HOT
CPUレベル	0	



技術仕様 (Blockly_CHaser_Tool.html)

Blockly_CHaser_Tool.html

⑤ マップデータをダウンロード

設定が完了したら「設定ファイル生成」ボタンをクリックしてマップデータをダウンロードします。

A screenshot of the Blockly_CHaser_Tool.html interface. At the top, there are three tabs: 'CPUターン' (CPU Turn), 'COOL', and 'HOT'. The 'HOT' tab is currently selected and highlighted in blue. Below the tabs, there is a label 'CPUレベル' (CPU Level) followed by a horizontal line and the number '1'. At the bottom right of the interface, there is a red button with the text '設定ファイル生成' (Generate Setting File).



技術仕様 (Blockly_CHaser_Tool.html)

Blockly_CHaser_Tool.html

- ⑥ 作成したマップを所定のフォルダに格納した後、ゲームサーバを再起動すると、マップが追加されます。

格納先フォルダは以下の通り。

CHaserGameServer

→ load_data

→ game_server_data



技術仕様 (Client.dll)

Targetクラスのプロパティ①

- Server_Address

接続先のゲームサーバURLを指定します。

例) `http://localhost:3000/`

[localhost]の部分を接続先サーバ名またはIPアドレスに変更することで接続先ゲームサーバを変更できます。

ただし接続ポートは[3000]固定です。

- Room_Name

接続するゲームサーバのマップIDを指定します。



技術仕様 (Client.dll)

Targetクラスのプロパティ②

- Name
ゲームサーバに表示されるプレイヤー名を指定します。
- First_Attack
先攻である場合は true、後攻である場合は false が設定されます。
注意①. CPU対戦の場合は必ず false が設定されます。
注意②. 一番最初にGetReady命令を実行した後に取得してください。



索引

索引

<u>if構文</u>	<u>... p.41</u>
<u>Look命令</u>	<u>... p.29</u>
<u>Put命令</u>	<u>... p.35</u>
<u>result</u>	<u>... p.25</u>
<u>Search命令</u>	<u>... p.32</u>
<u>switch構文</u>	<u>... p.47</u>
<u>Targetクラス</u>	<u>... p.85</u>
<u>value</u>	<u>... p.24</u>

<u>Walk命令</u>	<u>... p.26</u>
<u>配列</u>	<u>... p.72</u>
<u>変数</u>	<u>... p.64</u>
<u>大会ルール</u>	<u>... p.58</u>
<u>データ型</u>	<u>... p.66</u>
<u>デバッグ実行</u>	<u>... p.73</u>
<u>比較演算子</u>	<u>... p.44</u>
<u>マップ作成ツール</u>	<u>... p.78</u>



U-15 長野
プログラミング
コンテスト

お問合せ先

ご不明な点は以下までお問合せ下さい。

お問合せ先

U-15 長野プログラミングコンテスト実行委員会事務局

長野商工会議所内 TEL：026-227-2428 担当：小林 万里子